

DOCUMENTO

Introdução ao C++ para Competições de Programação

introducao-cpp.md

GERADO EM
23/08/2026 às 16:09

Sumário

Introdução ao C++ para Competições de Programação	3
1. O esqueleto de todo programa	3
O que cada linha faz	3
2. Entrada e saída	4
Lendo linhas inteiras	4
Lendo até o fim da entrada	5
Controlando casas decimais	5
3. Tipos de dados	5
Declaração e constantes	6
4. Operadores	6
5. Estruturas condicionais	6
6. Laços de repetição	7
7. Vetores e matrizes (arrays estáticos)	8
8. Strings	8
9. Funções	9
Recursão	9
10. Structs	10
11. Erros clássicos em competição	10
12. Exemplo completo	11

Introdução ao C++ para Competições de Programação

Este documento cobre o básico da linguagem C++ no contexto de maratonas e olimpíadas de programação, principalmente a OBI. ☹️☹️

1. O esqueleto de todo programa

Praticamente todo código de competição começa igual. Este é o template base:

```
#include <bits/stdc++.h>
using namespace std;

int main () {
    ios_base::sync_with_stdio(false);
    cin.tie(NULL);

    // Seu código aqui

    return 0;
}
```

cpp

O que cada linha faz

`#include <bits/stdc++.h>` É um cabeçalho não-padrão (existe no GCC/g++) que inclui *todas* as bibliotecas padrão de uma vez: `<iostream>`, `<vector>`, `<algorithm>`, `<cmath>`, `<string>` e todo o resto. Em competição isso economiza tempo e evita esquecer um include. Fora de competição, prefira incluir só o que usa — o `bits/stdc++.h` deixa a compilação mais lenta e não é portátil (não funciona no compilador da Microsoft nem no Clang do macOS por padrão).

`using namespace std;` Tudo da biblioteca padrão vive dentro do *namespace* `std`. Sem essa linha você precisaria escrever `std::cin`, `std::cout`, `std::vector<int>` a cada uso. Com ela, basta `cin`, `cout`, `vector<int>`. Em projetos grandes essa prática é desaconselhada (pode causar conflito de nomes), mas em um arquivo único de competição é o padrão.

`ios_base::sync_with_stdio(false);` Por padrão, o C++ mantém os fluxos `cin/cout` sincronizados com os do C (`scanf/printf`), para que você possa misturar os dois. Essa sincronização custa caro. Desligá-la torna `cin/cout` muito mais rápidos — a diferença aparece em problemas com centenas de milhares de linhas de entrada.

ATENÇÃO

Depois de desligar a sincronização, **não misture** `cin/cout` com `scanf/printf` no mesmo programa: a ordem da saída pode sair embaralhada.

`cin.tie(NULL);` Por padrão, `cin` está “amarrado” a `cout`: antes de cada leitura, o `cout` é descarregado (*flush*) automaticamente. Isso garante que um prompt apareça antes da leitura, mas em competição não há usuário humano lendo prompts — então desamarrar elimina um flush a cada leitura e ganha mais velocidade.

`return 0;` Indica ao sistema que o programa terminou com sucesso. No `main` do C++ ele é opcional (o compilador o insere implicitamente), mas é bom hábito escrevê-lo.

2. Entrada e saída

`cin` lê da entrada padrão, `cout` escreve na saída padrão.

```
int n;
cin >> n;           // lê um inteiro

int a, b;
cin >> a >> b;      // lê dois inteiros (separados por espaço ou quebra de linha)

cout << "Soma: " << a + b << "\n";
```

cpp

- `>>` ignora espaços em branco e quebras de linha automaticamente.
- Prefira `"\n"` a `endl`. O `endl` além de quebrar linha força um *flush* do buffer, e milhares de flushes deixam o programa lento.

Lendo linhas inteiras

```
string linha;
getline(cin, linha); // lê até a quebra de linha, incluindo espaços
```

cpp

Cuidado com a armadilha clássica: se você usa `cin >> n` e depois `getline`, o `getline` lê o resto vazio da linha do `n`. Solução:

```
int n;
cin >> n;
cin.ignore(); // descarta o '\n' pendente
string linha;
getline(cin, linha);
```

Lendo até o fim da entrada

Muitos problemas não informam a quantidade de casos de teste:

```
int x;
while (cin >> x) {
    // processa x até a entrada acabar
}
```

cpp

Controlando casas decimais

```
double media = 7.3333;
cout << fixed << setprecision(2) << media << "\n"; // 7.33
```

cpp

3. Tipos de dados

Tipo	Tamanho	Faixa aproximada
int	4 bytes	$-2 \cdot 10^9$ a $2 \cdot 10^9$
long long	8 bytes	$-9 \cdot 10^{18}$ a $9 \cdot 10^{18}$
unsigned int	4 bytes	0 a $4 \cdot 10^9$
float	4 bytes	≈ 7 dígitos significativos
double	8 bytes	≈ 15 dígitos significativos
char	1 byte	um caractere ('a', 'Z', '0')
bool	1 byte	true / false
string	variável	texto

O erro mais comum em competições é o **overflow** de `int`. Se o resultado puder passar de 2 147 483 647, use `long long`:

```
long long soma = 0;
for (int i = 0; i < n; i++) soma += v[i];
```

cpp

Atenção: mesmo guardando em `long long`, a conta `a * b` com `a` e `b` do tipo `int` é feita em `int` e transborda **antes** de ser atribuída. Force o tipo:

```
long long p = (long long) a * b;
```

cpp

Declaração e constantes

```
int n = 10;
const int MAXN = 100005; // constante – não pode ser alterada
double pi = 3.14159265358979;
char c = 'A';
string nome = "Gustavo";
bool ok = true;
```

cpp

4. Operadores

Aritméticos: `+`, `-`, `*`, `/`, `%`

```
7 / 2    // 3  (divisão inteira!)
7.0 / 2  // 3.5 (basta um operando ser real)
7 % 2    // 1  (resto da divisão)
```

cpp

Relacionais: `==`, `!=`, `<`, `>`, `<=`, `>=` — resultam em `true/false`.

Lógicos: `&&` (e), `||` (ou), `!` (não).

Atribuição composta: `+=`, `-=`, `*=`, `/=`, `%=`.

Incremento/decremento: `i++`, `++i`, `i--`, `--i`.

Bit a bit: `&`, `|`, `^`, `~`, `<<`, `>>`. `x << 1` dobra `x`; `x >> 1` divide por 2; `x & 1` diz se `x` é ímpar.

5. Estruturas condicionais

```
if (nota >= 7.0) {  
    cout << "Aprovado\n";  
} else if (nota >= 5.0) {  
    cout << "Recuperacao\n";  
} else {  
    cout << "Reprovado\n";  
}
```

cpp

Operador ternário — um `if/else` em uma linha:

```
cout << (a > b ? a : b) << "\n"; // imprime o maior
```

cpp

`switch` para comparar um valor contra várias constantes:

```
switch (opcao) {  
    case 1: cout << "Um\n"; break;  
    case 2: cout << "Dois\n"; break;  
    default: cout << "Invalido\n";  
}
```

cpp

Sem o `break`, a execução “vaza” para o próximo caso.

6. Laços de repetição

```
// for: quando você sabe quantas iterações  
for (int i = 0; i < n; i++) {  
    cout << i << "\n";  
}  
  
// while: enquanto a condição for verdadeira  
while (n > 0) {  
    n /= 2;  
}  
  
// do-while: executa ao menos uma vez  
do {  
    cin >> x;  
} while (x != 0);
```

cpp

`break` sai do laço imediatamente; `continue` pula para a próxima iteração.

Laços aninhados são a base de matrizes e de algoritmos $O(n^2)$:

```
for (int i = 0; i < n; i++)  
    for (int j = 0; j < m; j++)  
        cout << mat[i][j] << " ";
```

cpp

7. Vetores e matrizes (arrays estáticos)

```
int v[100];           // 100 inteiros, índices de 0 a 99  
int w[5] = {1, 2, 3, 4, 5};  
int mat[100][100];   // matriz 100x100
```

cpp

Leitura típica:

```
int n;  
cin >> n;  
int v[n];           // funciona no g++, mas não é C++ padrão  
for (int i = 0; i < n; i++) cin >> v[i];
```

cpp

O mais seguro é declarar com tamanho máximo constante e **globalmente**:

```
const int MAXN = 100005;  
int v[MAXN];  
  
int main () {  
    // ...  
}
```

cpp

Arrays globais são zerados automaticamente; arrays locais contêm lixo. Além disso, arrays grandes declarados dentro do `main` podem estourar a pilha — declare-os fora dela.

Índices começam em 0 e vão até `tamanho - 1`. Acessar fora disso não gera erro de compilação, mas causa comportamento indefinido (o famoso *runtime error*).

8. Strings

```
string s = "olimpiada";
cout << s.size() << "\n";    // 9 – quantidade de caracteres
cout << s[0] << "\n";        // 'o' – acesso por índice
s += " brasileira";           // concatenação
```

cpp

Percorrendo caractere a caractere:

```
for (int i = 0; i < (int) s.size(); i++) {
    if (s[i] == 'a') cout << "achei um a na posicao " << i << "\n";
}
```

cpp

Caracteres são números na tabela ASCII, o que permite truques úteis:

```
char d = '7';
int valor = d - '0';    // 7
char maiuscula = 'a' - 32;    // 'A' (ou use toupper('a'))
```

cpp

Compare `s.size()` com um `int` usando `cast`: `s.size()` retorna um tipo sem sinal, e uma comparação com um `int` negativo dá resultado inesperado.

9. Funções

```
int soma (int a, int b) {
    return a + b;
}

void imprime (int x) {    // void = não retorna nada
    cout << x << "\n";
}

int main () {
    imprime(soma(2, 3));    // 5
}
```

cpp

A função precisa ser declarada **antes** de ser usada.

Recursão

```
long long fatorial (int n) {  
    if (n <= 1) return 1;           // caso base  
    return n * fatorial(n - 1);    // caso recursivo  
}
```

cpp

Toda função recursiva precisa de um caso base, senão a pilha estoura.

10. Structs

Agrupam dados relacionados sob um mesmo nome:

```
struct Aluno {  
    string nome;  
    int idade;  
    double nota;  
};  
  
int main () {  
    Aluno a;  
    a.nome = "Ana";  
    a.idade = 17;  
    a.nota = 9.5;  
  
    cout << a.nome << " tirou " << a.nota << "\n";  
}
```

cpp

11. Erros clássicos em competição

Sintoma	Causa provável
<i>Wrong Answer</i> só em casos grandes	overflow de <code>int</code> — troque por <code>long long</code>
<i>Time Limit Exceeded</i> com I/O pesado	faltou <code>sync_with_stdio(false)</code> ou usou <code>endl</code>
<i>Runtime Error</i>	acesso fora dos limites do array, divisão por zero, recursão infinita
<i>Presentation Error</i>	quebra de linha ou espaço a mais/a menos na saída
Comparação de <code>double</code> falhando	use <code>fabs(a - b) < 1e-9</code> em vez de <code>a == b</code>

NOTA**Outras dicas:**

- Leia o formato da saída com atenção: acentos, maiúsculas e a quebra de linha final importam.
- Não imprima textos como "Digite um numero:" — o juiz compara a saída caractere a caractere.
- Estime a complexidade antes de codar: um juiz típico executa da ordem de 10^8 operações simples por segundo.

12. Exemplo completo

Problema: ler n e depois n inteiros; imprimir a soma e a média com duas casas decimais.

```
#include <bits/stdc++.h>
using namespace std;

int main () {
    ios_base::sync_with_stdio(false);
    cin.tie(NULL);

    int n;
    cin >> n;

    long long soma = 0;
    for (int i = 0; i < n; i++) {
        int x;
        cin >> x;
        soma += x;
    }

    double media = (double) soma / n;

    cout << "Soma: " << soma << "\n";
    cout << "Media: " << fixed << setprecision(2) << media << "\n";

    return 0;
}
```

cpp